

Deployment Strategies and Rollback

Every strategy trades cost against blast radius. None of them is simply best.

You are always trading three things: **cost during deployment, time to deploy, and blast radius on failure.**

The strategies

STRATEGY	HOW IT WORKS	FAILURE COSTS YOU
All at once	Replace everything, immediately	Full outage until rollback
Rolling	Batch by batch, in place	Reduced capacity, mixed versions
Rolling with batch	Adds temporary instances first	Mixed versions, full capacity kept
Immutable	New instances, then swap	Terminate the new group. Nothing else
Blue/green	Whole parallel environment	Shift traffic back. Near instant
Canary	A slice of traffic first	Only that slice, briefly

ON THE EXAM

The last column is what the exam actually asks. Read the scenario for what a failure must not cost, not for what sounds modern.

Rolling or immutable?

ROLLING

Update in place, in batches

- No extra instances to pay for
- Two versions run at once
- A half-failed deploy leaves a mess
- Rollback means deploying again

IMMUTABLE

Build new, then swap

- Doubles instances during deploy
- Never mixes versions
- Failure leaves the old set untouched
- Rollback is terminating a group

THE TIEBREAKER

Mixed versions must never be visible, or rollback must be fast, means immutable. Cost sensitivity means rolling.

How traffic actually shifts

- ◆ **Weighted target groups** on an ALB shift a percentage between old and new. The usual mechanism.
- ◆ **Lambda alias weights** shift invocations between two function versions.
- ◆ **Route 53 weighted records** also work, and DNS caching means clients may ignore you for the TTL.
- ◆ **Bake time** is the pause after shifting, during which alarms can still trigger a rollback.

ON THE EXAM

DNS-based shifting cannot guarantee when traffic moves. Load balancer and alias weights take effect immediately.

CodeDeploy

- ◆ **In-place or blue/green.** EC2 and on-premises can do either; Lambda and ECS are always blue/green.
- ◆ **Canary** shifts a percentage, waits, then the rest. **Linear** shifts equal increments on a timer.
- ◆ **Lifecycle hooks** run Lambda functions at defined points, and a failing hook triggers a rollback.
- ◆ **Automatic rollback on a CloudWatch alarm** is the feature that makes the rest of it safe.

ON THE EXAM

Named configurations like `Canary10Percent5Minutes` encode the whole strategy. If a scenario names percentages and minutes, this is it.

Containers: a changed recommendation

- ◆ **ECS rolling** is the default controller: replace tasks in place, governed by minimum and maximum healthy percent.
- ◆ **The CODE_DEPLOY controller** was the established way to get blue/green on ECS, with two target groups.
- ◆ **ECS now has built-in blue/green**, with lifecycle hooks, bake time and managed rollback. Canary and linear followed.
- ◆ AWS now recommends the **ECS-native** path for new deployments. Most study material still says CodeDeploy.

ON THE EXAM

Both are in scope and both are defensible. If a scenario names CodeDeploy, use it; if it asks for the simplest current approach, ECS-native.

Serverless deployment

- ◆ **Publish a version, point an alias at it**, and shift alias weights. Versions are immutable, which makes rollback trivial.
- ◆ **PreTraffic and PostTraffic hooks** run tests before and after the shift, and can fail the deployment.
- ◆ **Provisioned concurrency attaches to a version or alias**, so a new version starts cold unless you configure it.
- ◆ Alarms on errors, throttles and duration are what make automatic rollback meaningful.

ON THE EXAM

Rollback for Lambda is repointing an alias, which is close to instantaneous. Nothing is rebuilt or redeployed.

Elastic Beanstalk and Auto Scaling

- ◆ **Beanstalk has five policies:** all at once, rolling, rolling with additional batch, immutable, traffic splitting.
- ◆ **Rolling needs a manual redeploy to roll back.** Immutable and traffic splitting roll back on their own.
- ◆ Replacing all instances **loses accumulated EC2 burst balances**, which surprises people on burstable instance types.
- ◆ **Auto Scaling instance refresh** does the same job for a plain ASG, with checkpoints and a minimum healthy percentage.

ON THE EXAM

Beanstalk's traffic splitting is canary under another name: a second ASG, a percentage of traffic, and an evaluation period.

The pipeline around it

- ◆ **CodePipeline** orchestrates stages. **CodeBuild** builds and tests. **CodeDeploy** performs the release.
- ◆ **Manual approval actions** gate a stage, which is how a change reaches production only with a human decision.
- ◆ Cross-account deployment uses a role in the target account and a **KMS key shared with it** for the artefact bucket.
- ◆ Test in the pipeline, not after it. A deployment strategy is a safety net, not a substitute for tests.

ON THE EXAM

The classic cross-account pipeline failure is the artefact bucket's KMS key not being shared with the target account.

Reading the scenario

THE SCENARIO SAYS

Two versions must never run at once

Rollback in seconds, cost is acceptable

Catch regressions on real traffic first

No extra capacity may be provisioned

A human must approve production

Roll back automatically on errors

IT IS POINTING AT

Immutable or blue/green

Blue/green

Canary or traffic splitting

Rolling

Manual approval action

CloudWatch alarm plus bake time

ON THE EXAM

The constraint sentence is usually about money or about mixed versions. Find it before you compare the options.

Meridian Freight's tracking API must be updated weekly with no downtime. A bad release must be reversible within one minute. The platform runs on EC2 behind an Application Load Balancer, and the business accepts doubled compute cost for the duration of a deployment. The current rolling deployment takes 40 minutes and rollback takes another 40.

What should the solutions architect recommend?

- ☐ A Reduce the rolling deployment batch size so failures affect fewer instances
- ☐ B A blue/green deployment with weighted target groups and CodeDeploy
- ☐ C An immutable deployment
- ☐ D All at once, during a scheduled maintenance window

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

A Smaller batches

Makes each batch safer and the whole deployment longer. Rollback is still another full deployment, so one minute is impossible.

C Immutable

The closest miss. Rollback is fast once the swap has happened, and after the old group is terminated you are redeploying again.

B Blue/green with weighted target groups

The old environment stays running, so rollback is shifting weights back. Seconds, not minutes.

D All at once

Directly contradicts no downtime, and a maintenance window does not make an outage into not-an-outage.

THE SIGNAL

One minute to reverse is the constraint. Only a strategy that keeps the old environment running can meet it.

A rolling deployment failed at batch three of six. Half the fleet runs the new version and half the old, and a schema change means the two are not compatible. The team spent two hours restoring service.

They want failures to leave the environment in a known state, and cannot justify permanently doubling capacity.

What should the solutions architect recommend?

- ☐ A Rolling with an additional batch, to maintain full capacity
- ☐ B An immutable deployment
- ☐ C All at once, so both versions never coexist
- ☐ D Increase the health check grace period so batches are validated for longer

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A

 Rolling with additional batch
Fixes capacity during deployment and still mixes versions batch by batch, which is the actual failure described.
- B

 Immutable
Builds a complete new set and swaps only when healthy. Versions never mix, and a failure terminates the new group.
- C

 All at once
True that versions never coexist, achieved through an outage on every release. It also has no rollback beyond redeploying.
- D

 Longer grace period
Validates each batch more thoroughly, and does nothing about the incompatibility between batches that have deployed and those that have not.

THE SIGNAL

Doubling capacity permanently is not the same as doubling it during a deployment. Read which one the scenario refuses.

A Lambda function behind an API handles 40 million invocations a day. A recent release introduced an error affecting roughly 2% of requests, which passed all pre-production tests and was only noticed after four hours.

The team wants real traffic to validate a release, with automatic reversal.

What should the solutions architect implement?

- ☐ A CodeDeploy canary traffic shifting on the function alias, with a CloudWatch alarm on the error metric
- ☐ B Increase pre-production test coverage and add a staging environment
- ☐ C Deploy to a second function and switch the API Gateway integration when confident
- ☐ D Enable provisioned concurrency on the new version

◆ PAUSE AND DECIDE

What should the solutions architect implement?

- A

Canary shifting plus alarm
A small slice of real traffic, an alarm watching errors, and automatic reversal by repointing the alias. Every requirement met.
- B

More testing
The bug already passed all pre-production tests. More of the thing that missed it is not a plan.
- C

Second function, manual switch
Builds blue/green by hand with a human deciding when. The requirement said automatic.
- D

Provisioned concurrency
Addresses cold starts, which is a latency concern. It has no bearing on functional errors.

THE SIGNAL

Passed all pre-production tests is the planted detail. It exists to eliminate every answer that adds more testing.

A team is building a new ECS service on Fargate behind an Application Load Balancer, and wants blue/green deployments with validation hooks and automatic rollback. They have no existing CodeDeploy configuration and no preference.

They ask which approach involves the fewest moving parts.

What should the solutions architect recommend?

- A The CODE_DEPLOY deployment controller, with CodeDeploy managing two target groups
- B The built-in ECS blue/green deployment strategy with ECS lifecycle hooks
- C The default ECS rolling controller with a low minimum healthy percent
- D The EXTERNAL deployment controller with a custom deployment system

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A

CODE_DEPLOY controller
The established answer and still fully supported. It adds a second service to configure, which the fewest-moving-parts requirement counts against.
- B

ECS built-in blue/green
Hooks, bake time and managed rollback inside ECS itself, with no CodeDeploy application or deployment group to maintain.
- C

Rolling controller
Replaces tasks in place, so it is not blue/green and offers none of the validation the team asked for.
- D

EXTERNAL controller
Hands the entire deployment process to software you write. The opposite of fewest moving parts.

THE SIGNAL

This answer changed in 2025. Material written before then teaches option A, and option A is still correct where CodeDeploy already exists.

A CodePipeline in a tooling account deploys to a production account. The source and build stages succeed. The deploy stage fails with an access denied error when the target account tries to read the artefact from S3.

The cross-account role exists and has S3 read permission on the artefact bucket.

What is the most likely cause?

- ☐ A The KMS key encrypting the artefact bucket is not shared with the production account
- ☐ B The pipeline needs to run in the production account instead
- ☐ C The artefact bucket policy must allow s3:ListBucket as well
- ☐ D CodePipeline does not support cross-account deployment

◆ PAUSE AND DECIDE

What is the most likely cause?

- A

KMS key not shared
The artefact bucket is encrypted with a customer managed key, and reading an object needs Decrypt on that key as well as S3 access.
- B

Run it locally
Abandons the separation of tooling and production that the design deliberately has. Cross-account pipelines are the recommended pattern.
- C

Add ListBucket
Needed for listing, not for reading a known object key. It would not produce this failure.
- D

Not supported
It is explicitly supported and widely used. Two stages already succeeded.

THE SIGNAL

This is module two's third door reappearing. S3 access and KMS access are two separate authorisation decisions.

A blue/green deployment completed successfully. All health checks passed, traffic shifted to green, and the blue environment was terminated. Twenty minutes later, error rates climbed as a memory leak in the new version began to bite.

The team wants this class of failure caught automatically in future.

What should the solutions architect change?

- ☐ A Add deeper health checks on the target group
- ☐ B Configure a bake time before terminating blue, with a CloudWatch alarm on error rate as a rollback trigger
- ☐ C Switch to a rolling deployment so fewer instances run the new version
- ☐ D Run load tests in pre-production before every release

◆ PAUSE AND DECIDE

What should the solutions architect change?

A Deeper health checks

Health checks pass or fail at a moment. A leak that takes twenty minutes to matter looks healthy at every check until it does not.

C Rolling deployment

Reduces how many instances run the new version, and every instance eventually runs it. The leak arrives regardless, more slowly.

B Bake time plus alarm rollback

Keeps blue alive through a monitoring window, so an alarm firing at twenty minutes can still shift traffic back instantly.

D Pre-production load tests

Worth doing and not automatic detection in production. The requirement was to catch this class of failure automatically.

THE SIGNAL

Terminating blue immediately is the mistake. A rollback plan that has already deleted its target is not a rollback plan.

What to carry out of this module

- ◆ **Find the constraint sentence.** It is usually about extra cost or about mixed versions, and it eliminates half the options.
- ◆ **Immutable doubles capacity during a deployment, not permanently.** Scenarios test whether you read that correctly.
- ◆ **Bake time plus a CloudWatch alarm** is what turns a deployment strategy into automatic rollback.
- ◆ ECS blue/green is now native as well as CodeDeploy. Cross-account pipelines fail on the KMS key.

ON THE EXAM

A rollback plan that terminates the old environment immediately is not a rollback plan. Keep it alive for the monitoring window.